DBMS LAB — QUESTION 4

AIM

Implement aggregate functions — SUM, AVG, MIN, MAX and COUNT — using GROUP BY and HAVING clauses.

DESCRIPTION

Aggregate functions calculate values from multiple rows. COUNT counts rows, SUM gives a total, AVG gives an average, MIN gives the smallest value and MAX gives the largest value.

GROUP BY forms groups and HAVING filters those groups.

DATABASE SETUP

```
CREATE TABLE Employee (
    EmpID NUMBER PRIMARY KEY, EmpName VARCHAR2(50),
    Department VARCHAR2(30), Salary NUMBER(10,2),
    City VARCHAR2(30)
);
INSERT INTO Employee VALUES (201,'John','HR',50000,'Hyderabad');
INSERT INTO Employee VALUES (202,'Alice','IT',60000,'Chennai');
INSERT INTO Employee VALUES (203,'Bob','Finance',55000,'Bangalore');
INSERT INTO Employee VALUES (204,'Emma','IT',65000,'Pune');
INSERT INTO Employee VALUES (205,'David','Marketing',48000,'Delhi');
INSERT INTO Employee VALUES (206,'Sophia','IT',70000,'Chennai');
COMMIT;
```

CODE 1 — COUNT TOTAL

```
SELECT COUNT(*) AS Total_Employees
FROM Employee;
```

EXPECTED OUTPUT

```
6
```

CODE 2 — COUNT BY DEPARTMENT

```
SELECT Department, COUNT(*) AS Employee_Count
FROM Employee
GROUP BY Department;
```

EXPECTED OUTPUT

DEPARTMENT	COUNT
HR	1
IT	3
Finance	1
Marketing	1

CODE 3 — SUM TOTAL

```
SELECT SUM(Salary) AS Total_Salary
FROM Employee;
```

EXPECTED OUTPUT

```
348000
```

CODE 4 — SUM BY DEPARTMENT

```
SELECT Department, SUM(Salary) AS Total_Salary
FROM Employee
GROUP BY Department;
```

EXPECTED OUTPUT

DEPARTMENT	SUM
HR	50000
IT	195000
Finance	55000
Marketing	48000

CODE 5 — AVG TOTAL

```
SELECT AVG(Salary) AS Average_Salary
FROM Employee;
```

EXPECTED OUTPUT

```
58000
```

CODE 6 — AVG BY DEPARTMENT

```
SELECT Department, AVG(Salary) AS Average_Salary
FROM Employee
GROUP BY Department;
```

EXPECTED OUTPUT

DEPARTMENT	AVG
HR	50000
IT	65000
Finance	55000
Marketing	48000

CODE 7 — MINIMUM

```
SELECT MIN(Salary) AS Minimum_Salary
FROM Employee;
```

EXPECTED OUTPUT

```
48000
```

CODE 8 — MINIMUM BY DEPARTMENT

```
SELECT Department, MIN(Salary) AS Minimum_Salary
FROM Employee
GROUP BY Department;
```

EXPECTED OUTPUT

DEPARTMENT	MIN
HR	50000
IT	60000
Finance	55000
Marketing	48000

CODE 9 — MAXIMUM

```
SELECT MAX(Salary) AS Maximum_Salary
FROM Employee;
```

EXPECTED OUTPUT

```
70000
```

CODE 10 — MAXIMUM BY DEPARTMENT

```
SELECT Department, MAX(Salary) AS Maximum_Salary
FROM Employee
GROUP BY Department;
```

EXPECTED OUTPUT

DEPARTMENT	MAX
HR	50000
IT	70000
Finance	55000
Marketing	48000

CODE 11 — ALL FUNCTIONS TOGETHER

```
SELECT Department, COUNT(*) AS Employee_Count,
SUM(Salary) AS Total_Salary, AVG(Salary) AS Average_Salary,
MIN(Salary) AS Minimum_Salary, MAX(Salary) AS Maximum_Salary
FROM Employee GROUP BY Department;
```

EXPECTED OUTPUT

DEPT	COUNT	SUM	AVG	MIN	MAX
HR	1	50000	50000	50000	50000
IT	3	195000	65000	60000	70000
Finance	1	55000	55000	55000	55000
Marketing	1	48000	48000	48000	48000

CODE 12 — HAVING WITH COUNT

```
SELECT Department, COUNT(*) AS Employee_Count
FROM Employee GROUP BY Department
HAVING COUNT(*) > 1;
```

EXPECTED OUTPUT

DEPARTMENT	COUNT
IT	3

CODE 13 — HAVING WITH SUM

```
SELECT Department, SUM(Salary) AS Total_Salary
FROM Employee GROUP BY Department
HAVING SUM(Salary) > 100000;
```

EXPECTED OUTPUT

DEPARTMENT	SUM
IT	195000

CODE 14 — HAVING WITH AVG

```
SELECT Department, AVG(Salary) AS Average_Salary
FROM Employee GROUP BY Department
HAVING AVG(Salary) > 55000;
```

EXPECTED OUTPUT

DEPARTMENT	AVG
IT	65000

CODE 15 — WHERE + GROUP BY + HAVING

```
SELECT Department,
       COUNT(*) AS Employee_Count,
       AVG(Salary) AS Average_Salary
FROM Employee
WHERE Salary >= 50000
```

```
GROUP BY Department
HAVING COUNT(*) >= 1;
```

EXPECTED OUTPUT

DEPARTMENT	EMPLOYEE_COUNT	AVERAGE_SALARY
Finance	1	55000
HR	1	50000
IT	3	65000

IMPORTANT POINTS

1. COUNT(*) counts rows. COUNT(column) counts non-NULL values in that column.

2. SUM and AVG are used with numeric columns.

3. GROUP BY creates separate results for each group.

4. HAVING filters grouped/aggregate results.

5. In Code 15, rows with Salary below 50000 are removed first by WHERE. Therefore David from Marketing (48000) is excluded from the grouped result.